

บทที่ 5

อาร์เรย์ (Array)



คณาจารย์ภาควิชาวิศวกรรมคอมพิวเตอร์
introcom@coe.psu.ac.th

วัตถุประสงค์

- สามารถใช้งานตัวแปรประเภทอาร์เรย์ 1 มิติและ 2 มิติได้
- เข้าใจการส่งผ่านอาร์เรย์ไปยังฟังก์ชัน
- เรียนรู้วิธีการเก็บข้อมูลสตริงและฟังก์ชันเกี่ยวกับสตริงที่น่าสนใจ



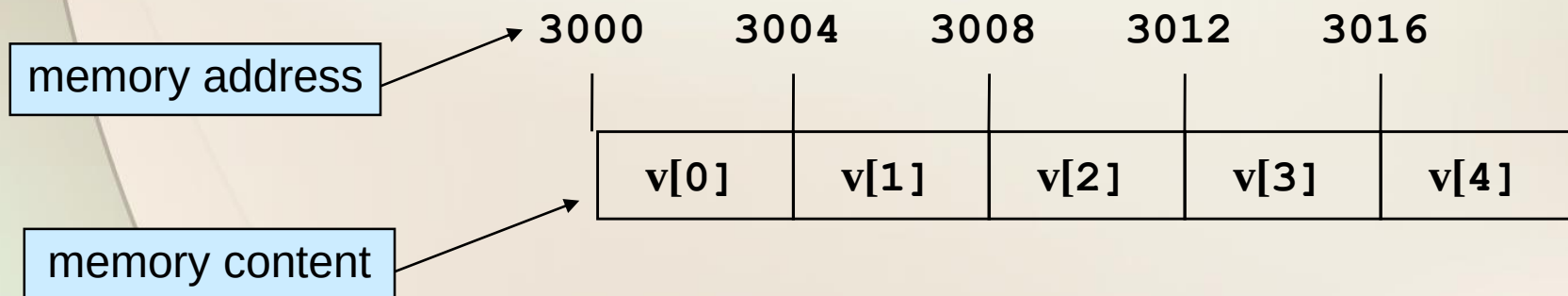
เนื้อหาในบทเรียน



- นิยามของอาร์เรย์
- ตัวแปรอาร์เรย์ แบบ 1 มิติ
- การเข้าถึงค่าในอาร์เรย์
- การรับและแสดงค่าอาร์เรย์
- การให้ค่าเริ่มต้นกับอาร์เรย์
- การส่งผ่านอาร์เรย์ไปยังฟังก์ชัน
- ตัวแปรอาร์เรย์แบบหลายมิติ
- สายตัวอักษร (String)

นิยามของอาร์เรย์

- ตัวแปรชุด หรือ อาร์เรย์ (Array) คือกลุ่มข้อมูลที่ประกอบไปด้วยข้อมูลชนิดเดียวกัน เรียงต่อเนื่องกันไปเป็นกลุ่ม โดยจัดอยู่ในบล็อกของหน่วยความจำเดียวกัน และใช้ชื่อตัวแปรร่วมกันในการอ้างอิง และมีดัชนี หรือ *index* ในการอ้างอิงข้อมูลแต่ละตัว
- ข้อมูลแต่ละตัวเก็บอยู่ในบล็อกของหน่วยความจำ เรียกว่า อีลีเมนต์ (Element)



อาร์เรย์แบบ 1 มิติ

อาร์เรย์หนึ่งมิติ มีโครงสร้างเทียบเท่าเมตริกซ์ขนาด $n \times 1$ การประกาศตัวแปรอาร์เรย์ จะใช้เครื่องหมาย `[ ]` ล้อมค่าตัวเลขจำนวนเต็ม เพื่อบอกจำนวนหน่วยข้อมูลที่ต้องการได้ในรูป

ชนิดของตัวแปร ชื่อตัวแปร[จำนวนสมาชิกที่ต้องการ]

data_type variable_name [number-of-elements]

เช่น `int      a[5];`
 `double   x, y[10], z[3];`

Note: จำนวนสมาชิกที่ต้องการ(ขนาดของอาร์เรย์) ต้องระบุเป็นค่าคงที่
 เท่านั้น(จำนวนนับ) เป็นตัวแปรหรือนิพจน์ไม่ได้

ขนาดของตัวแปรอาร์เรย์ และความจุในหน่วยไบต์

- เราสามารถหาความจุของอาร์เรย์ที่ถูกจัดเก็บที่หน่วยความจำ (memory) ในหน่วยไบต์ได้โดยใช้ `sizeof(ชื่อตัวแปร)`

ตัวอย่างเช่น

ขนาดหรือความยาวของอาร์เรย์

`int numbers[10];`

`cout<<sizeof (numbers) ;`

- ขนาดความจุ(ไบต์)ของตัวแปรชนิดอาร์เรย์ชื่อ `numbers` คือ

$$10 * 4 = 40 \text{ ไบต์}$$

ขนาดของอาร์เรย์

ขนาดของตัวแปรแบบ `int`

ตัวอย่างที่ 1 : การดูขนาดอาร์เรย์ 1 มิติในหน่วยไบต์

- ไฟล์ arrayex1.c

```
//arrayex1.c
```

```
int main()
```

```
{    int ages[10];  
    char name[50];  
    double scores[20];  
    cout<<"Size of ages = "<<sizeof(ages)<<endl;  
    cout<<"Size of name = "<<sizeof(name) <<endl;  
    cout<<"Size of scores = "<<sizeof(scores) <<endl;  
    cout<<"Size of name[0]= "<<sizeof(name[0]) <<endl;  
    cout<<"Size of ages[0]= "<<sizeof(ages[0]) <<endl;  
}
```

Size of ages = 40

Size of name = 50

Size of scores = 160

Size of name[0] = 1

Size of ages[0] = 4

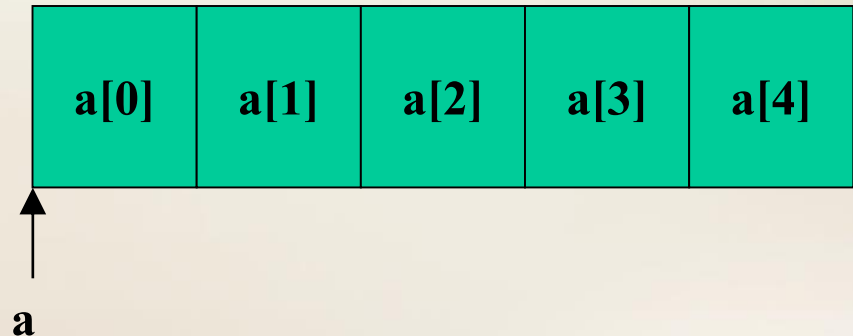
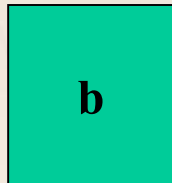
ข้อควรรู้เกี่ยวกับตัวแปรอาร์เรย์

การอ้างชื่อของตัวแปรอาร์เรย์เพียงอย่างเดียว จะหมายถึง ค่าตำแหน่งแอดเดรสของหน่วยความจำเริ่มต้นของกลุ่มข้อมูลอาร์เรย์

เช่น

```
int b;
```

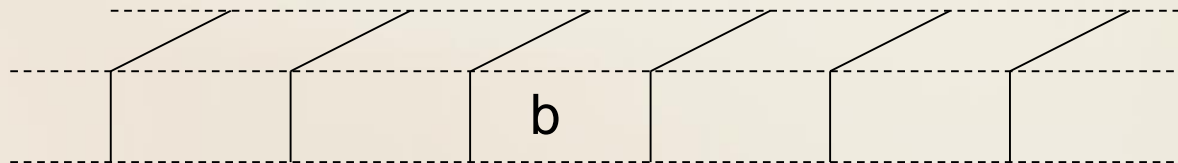
```
int a[5];
```



แอดเดรส (Address)

เมื่อกำหนดตัวแปร `int b`;

คอมพิวเตอร์จะแทนตัวแปร `b` ด้วยตำแหน่งหน่วยความจำ
เปรียบได้กับชื่อบ้านที่ใช้เรียกแทนบ้านเลขที่ เช่น บ้านเลขที่ 36 อาจ
เรียกว่า บ้านนายแดง ซึ่งเปรียบเหมือนชื่อ `b` ที่ใช้เรียกแทน
หน่วยความจำ นั้นเอง



แอดเดรส (Address)

หากต้องการทราบค่าตำแหน่งหน่วยความจำ ทำได้โดยการใส่เครื่องหมาย & นำหน้าชื่อตัวแปร ดังนั้นค่าแอดเดรสของตัวแปร b ก็คือ &b

เช่น การส่งค่า address ของตัวแปรให้กับคำสั่ง scanf ในการรับค่า

```
cout<<&b;
```

สรุป

แอดเดรส (Address) คือ ค่าที่ใช้อ้างถึงตัวข้อมูลภายในหน่วยความจำ เหมือนกับหมายเลขบ้านเลขที่

การเข้าถึงตัวแปรอาร์เรย์

- เมื่อมีการประกาศอาร์เรย์แล้ว ค่าตำแหน่งหมายเลขลำดับข้อมูล
สำหรับใช้เข้าถึงตัวแปรย่อยต่างๆ ในอาร์เรย์ จะถูกกำหนดโดย
อัตโนมัติ
- โดยหากกำหนดอาร์เรย์ด้วยขนาด n ข้อมูล หน่วยแรก จะมีค่า
ตำแหน่งลำดับเป็น 0 ไปจนถึงข้อมูลหน่วยสุดท้ายจะมีค่าตำแหน่ง
ลำดับเป็น $n-1$

เช่น

`int v[5];`

3000 3004 3008 3012 3016

v[0]	v[1]	v[2]	v[3]	v[4]

การเข้าถึงตัวแปรอาร์เรย์

- ถ้าต้องการอ่านหรือเขียนข้อมูลในหน่วยต่างๆ ของตัวแปรอาร์เรย์ จะต้องอ้างชื่อตัวแปรตามด้วยค่าลำดับของหน่วยในกลุ่มข้อมูลอาร์เรย์ ล้อมด้วยเครื่องหมาย [] ซึ่งเรียกว่า subscript (หรือดัชนี index)
- ค่าดัชนี อาจอยู่ในรูป ค่าคงที่ ของตัวแปร นิพจน์ หรือฟังก์ชันที่ให้ค่า เป็นค่าจำนวนเต็มก็ได้ (`positive integer >=0`)
- ของเขตของ index หรือ subscript มีค่าตั้งแต่ 0 ถึง $n-1$ (n คือขนาด ของอาร์เรย์)

ตัวอย่างการเข้าถึงตัวแปรอาร์เรย์

เช่น

```
int grades[5];  
  
grades[0] = 98;  
grades[1] = grades[0] - 11;  
grades[2] = 2 * (grades[0] - 6);  
grades[3] = 79;  
grades[4] = (grades[2]+grades[3]- 3)/2;  
total = grades[0]+ grades[1]+ grades[2]+  
        grades[3]+ grades[4];  
  
grades[i];  
grades[2*i];  
grades[j-i];
```

ตัวอย่างการใช้คำสั่งวนรอบ *for* กับอาร์เรย์

ถ้าต้องการหาผลรวมของตัวแปร grade ทั้ง 5 อีลีเมนต์ ทำดังนี้

```
total = grades[0] + grades[1] + grades[2] + grades[3] +  
grades[4];
```

เปลี่ยนเป็น for loop ได้ดังนี้

```
total = 0;  
for ( i = 0 ; i <= 4 ; i++)  
    total += grades[i];
```

การรับค่าและการแสดงค่าอาร์เรย์ (Input and Output)

ตัวอย่างการรับค่า

```
price[5] = 10.69;
```

```
cin>>grades[0];
```

```
cin>>price[2];
```

```
int price[5];
```

```
cin>>price[5];
```

เกิดอะไรขึ้น ???????

```
int a[?];
```

```
for( int count = 0 ; count < 5 ; count++)
```

```
    cin>>a[count];
```

การรับค่าและการแสดงค่าอาร์เรย์ (Input and Output)

ตัวอย่างการแสดงค่า

```
cout<<price[6];
```

```
cout<<"The value of element"<<grade[i];
```

```
for( n = 5 ; n <= 20 ; n++)
```

```
    cout<<"["<< n <<"]"<< price[n]<<endl;
```

ข้อควรระวัง ! **index** ของอาร์เรย์ ต้องมีค่าไม่เกิน

ขนาดของอาร์เรย์-1

การใช้คำสั่งวนรอบ *for* ในการเข้าถึงค่าในอาร์เรย์

เราสามารถ ใช้คำสั่งวนรอบ **for** ในการวนรอบรับค่าที่ป้อนเข้ามาและใช้ในการคำนวณ โดยการใช้ตัวแปรในการวนรอบ และใช้ตัวแปรเดียวกันเพื่อกำหนดลำดับของข้อมูลที่จะใช้ในอาร์เรย์

```
int x,a[5];
for (x=0; x<5; x++)
{ cout<<"Enter value for a["<<x<< "]:");
  cin>>a[x];
}
cout<<"Show all values\n";
for (x=0; x<5; x++)
{ cout<< "a[" <<x<< "]" = "<<a[x];
}
```

ตัวอย่างที่ 2 : การรับค่าและแสดงค่าของอาร์เรย์

- ไฟล์ arrayex2.c

```
#include <iostream>
int main()
{
    int num[4],i;
    for(int i=0;i<4;i++)
    {
        cout<<"Enter num ["<< i <<"]:";
        cin>>num[i];
    }
    for(i=0;i<4;i++)
        num[i] = 2*num[i];
    for(i=0;i<4;i++)
        cout<<"num ["<< i <<"]:"<<num[i]<<endl;
    return 0;
}
```

Enter num[0]: 10

Enter num[1]: 20

Enter num[2]: 30

Enter num[3]: 40

num[0] = 20

num[1] = 40

num[2] = 60

num[3] = 80



การให้ค่าเริ่มต้น (Array Initialization)

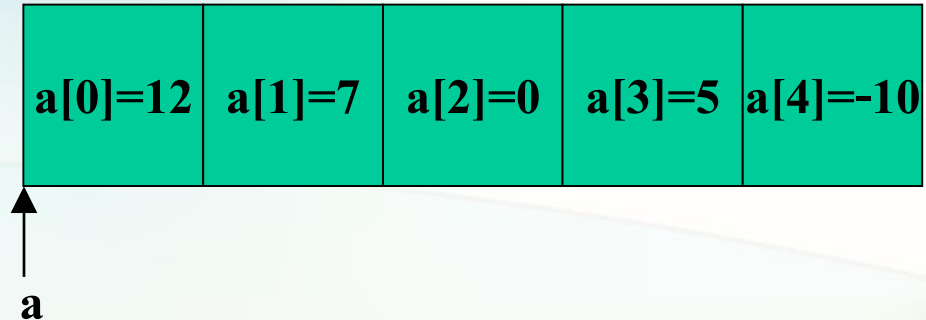
ในการกำหนดค่าเริ่มต้นให้กับตัวแปรอาร์เรย์นั้น สามารถกำหนดไปพร้อมกับการกำหนดตัวแปรได้เช่นเดียวกันกับตัวแปรเดี่ยว แต่จะใช้กลุ่มของค่าคงที่ในการกำหนดค่าเริ่มต้น

ชนิดของตัวแปรอาร์เรย์ ชื่ออาร์เรย์[จำนวนข้อมูล] = {ค่าคงที่,ค่าคงที่,...};

ตัวอย่างการให้ค่าเริ่มต้น(Array Initialization)

เช่น

```
int a[5] = {12,7,0,5,-10}
```



```
char codes[6] = {'s', 'a', 'm', 'p', 'l', 'e'};
```

```
double width[7] = {10.96, 6.43, 2.58, 0.86, 5.89, 7.56, 8.22};
```

```
float temp[4] = {98.6, 97.2, 99.0 , 101.5};
```

```
int gallons[20] = { 19, 16, 14, 19, 20,
                   18, 12, 10, 22, 15,
                   18, 17, 16, 24, 23,
                   19, 15, 18, 21, 5  };
```

การให้ค่าเริ่มต้น (Array Initialization)

- ค่าคงที่ที่ใช้ในการกำหนดค่าเริ่มต้น จะต้องมีชนิดสอดคล้องกับชนิดอาร์เรย์ ค่าคงที่แต่ละค่าจะถูกนำไปกำหนดให้กับสมาชิกแต่ละตัวตามลำดับ
- หากเราใช้ค่าคงที่จำนวนน้อยกว่าสมาชิกของอาร์เรย์ ตำแหน่งที่เหลือจะถูกกำหนดให้เป็น 0

เช่น

```
int a[5] = {12 , 7 };
```

มีค่าเท่ากับ

```
int a[5] = {12 , 7 , 0 , 0 , 0 };
```

การให้ค่าเริ่มต้น (Array Initialization)

การประกาศตัวแปรแบบอาร์เรย์พร้อมทั้งกำหนดค่าเริ่มต้น สามารถทำได้ โดยไม่จำเป็นต้องกำหนดขนาดของอาร์เรย์ก็ได้ คอมไพเลอร์จะหาจำนวนสมาชิกในกลุ่มค่าคงที่เอง เช่น

```
char codes[6] = {'s', 'a', 'm', 'p', 'l', 'e'};
char codes[] = {'s', 'a', 'm', 'p', 'l', 'e'};
```

ทั้งสองบรรทัดทำงานเหมือนกัน โดยคอมไพเลอร์จะรู้ได้เองว่าขนาดของอาร์เรย์จะมีขนาดเป็น 6 แต่ห้ามเว้นว่างไว้โดยไม่กำหนดอะไรเลย

เช่น

~~char codes[];~~

การให้ค่าเริ่มต้น (Array Initialization)

ในกรณีให้ค่าเริ่มต้นแก่ตัวแปรอาร์เรย์แบบ char สามารถทำได้ โดยใช้สตริงได้ด้วย ดังนี้

```
char code[] = "sample";
```

คอมไพเลอร์จะใส่ ศูนย์ หรือ ‘\0’ หรือเรียกว่า *null character* ไว้ท้ายข้อความเสมอ

codes[0] codes[1] codes[2] codes[3] codes[4] codes[5] codes[6]

s	a	m	p	l	e	\0
---	---	---	---	---	---	----

ตัวอย่างที่ 3 : โปรแกรมสำหรับหาค่าสูงสุดในอาร์เรย์

- ไฟล์ arrayex3.c

```
#include <iostream>
int main()
{
    int a[4]={-1,6,9,2};
    int max = a[0];
    for(int i=1;i<4;i++){
        if(a[i] > max)
            max = a[i];
    }
    cout<<"Maximum value is ",max<<endl;
    return 0;
}
```

Maximum value is 9

ตัวอย่างที่ 4 : โปรแกรมสำหรับหาค่าผลบวกในอาร์เรย์

- ไฟล์ arrayex4.c

```
#include <iostream>
#define SIZE 4
int main()
{
    int num[SIZE] = {1,4,5,7};
    int total = 0;
    for(int i=0;i<SIZE;i++)
        total = total + num[i];
    cout<<"Sum of all elements =" <<total<<endl;
    return 0;
}
```

Sum of all elements = 17



การส่งผ่านอาร์เรย์ไปยังฟังก์ชัน

สามารถแบ่งได้เป็น 2 ลักษณะ

- การส่งผ่านค่าอีลีเมนต์อาร์เรย์ให้กับฟังก์ชัน เป็นการเรียกใช้ฟังก์ชันแบบ Call-by-value
- การส่งอาร์เรย์ทุกอีลีเมนต์ให้กับฟังก์ชัน เป็นการเรียกใช้ฟังก์ชันแบบ Call-by-reference

การเรียกใช้แบบ Call-by-value

- ใช้วิธีการส่งค่าของตัวแปร (value) ให้กับฟังก์ชัน โดยผ่านพารามิเตอร์
- ไม่สามารถแก้ไขค่าของอาร์กิวเมนต์(หรือพารามิเตอร์) ภายในฟังก์ชันได้ = **การแก้ไขค่าต่างๆในฟังก์ชัน ไม่มีผลต่อตัวแปรที่ส่งค่ามา**
- ใช้กับฟังก์ชันที่รับค่าเข้าเป็นตัวแปรธรรมดา (int, float, char,...)

- เช่น void triple(int x)

```
{ x=x*3;
```

```
cout<<"x = "<<x<<endl;
```

```
}.....
```

```
int x=5, y[2]={10,11};
```

```
triple(x); triple(y[0]);
```

triple(x) ส่งค่า **5** ให้กับฟังก์ชัน ในฟังก์ชัน **x** เริ่มต้นเป็น **5** และถูกทำให้กลายเป็น **15** หลังจบฟังก์ชัน ค่า **x** นอกฟังก์ชัน ไม่เปลี่ยนแปลง

triple(y[0]) ส่งค่า **10** ให้กับฟังก์ชัน ในฟังก์ชัน **x** เริ่มต้นเป็น **10** และถูกทำให้กลายเป็น **30** หลังจบฟังก์ชัน ค่า **y[0]** ยังเหมือนเดิม



การเรียกใช้แบบ Call-by-reference

- ใช้วิธีการส่งค่า แอดเดรส (Address)*** ของตัวแปรไปให้ฟังก์ชัน
 - ใช้กับฟังก์ชันที่รับค่าเข้าเป็นอาร์เรย์
 - สามารถแก้ไขค่าของอาร์กิวเมนต์ภายในฟังก์ชันได้ = การแก้ไขค่าตัวแปรอาร์เรย์ ภายในฟังก์ชัน มีผลการเปลี่ยนแปลงต่อตัวแปรที่ส่งค่ามา เพราะ การมีการจัดการค่าของหน่วยความจำในตำแหน่งเดียวกัน
- ***แอดเดรส (Address) คือ ค่าที่ใช้อ้างถึงตัวข้อมูลภายในหน่วยความจำ เหมือนกับหมายเลขบ้านเลขที่***

ฟังก์ชันที่มีการรับค่าเข้าเป็นอาร์เรย์

- ฟังก์ชันสามารถที่จะรับค่าเข้าเป็นอาร์เรย์ได้ ซึ่งรูปแบบของการเขียนต้นแบบของฟังก์ชันเป็นดังนี้

ชนิดข้อมูล ชื่อฟังก์ชัน(ชนิดข้อมูล ชื่อตัวแปร[ขนาดอาร์เรย์]);

- ในกรณีฟังก์ชันมีการรับค่าเข้าเป็นอาร์เรย์ 1 มิติ อาจจะไม่ต้องกำหนดขนาดของอาร์เรย์ก็ได้
- ตัวอย่างเช่น

```
int sum_arr(int num[10]);  
void print_arr(int a[5]);  
float average(int num[]);
```

การส่งผ่านค่าอีลีเมนต์อาร์เรย์ให้กับฟังก์ชัน

- หากฟังก์ชัน `my_func` มีต้นแบบของฟังก์ชันดังนี้

```
void my_func(int x);
```

- และใน `main` ได้มีการประกาศตัวแปรอาร์เรย์ชื่อว่า `num`

```
int num[10];
```

- การส่งอีลีเมนต์ที่ 0 ของอาร์เรย์ `num` ไปเป็นอาร์กิวเมนต์ของฟังก์ชัน

`my_func` สามารถเขียนได้ดังนี้

```
my_func(num[0]);
```

ตัวอย่างที่ 5 : การส่งค่าแต่ละอีลีเมนต์ในอาร์เรย์ให้กับฟังก์ชัน

- ไฟล์ arrayex5.c

```
#include <iostream>
void check_val(int x);
int main()
{
    int a[3] = {2, -1, 5};
    check_val(a[0]);
    return 0;
}
void check_val(int x)
{
    if(x >= 0)
        cout<<x<<" : Positive\n";
    else
        cout<<x<<" : Negative\n";
}
```

2 : Positive

ตัวอย่างที่ 6 : การส่งค่าแต่ละอีลีเมนต์ในอาร์เรย์ให้กับฟังก์ชัน

- ไฟล์ arrayex6.c

```
#include <iostream>
void check_val(int x);
int main()
{
    int a[3]={2,-1,5};
    for(int i=0;i<3;i++)
        check_val(a[i]);

    return 0;
}
void check_val(int x)
{
    if(x >= 0)
        cout<<x<<" : Positive\n";
    else
        cout<<x<<" : Negative\n";
}
```

2 : Positive
-1 : Negative
5 : Positive

การส่งอาร์เรย์ทุกอีลีเมนต์ของอาร์เรย์ให้กับฟังก์ชัน

- การส่งอาร์เรย์ในกรณีนี้ ใช้แค่ชื่อตัวแปรอาร์เรย์เท่านั้น เช่น หากใน main มีการประกาศอาร์เรย์ดังนี้

```
int num[10];
```

- และฟังก์ชัน print_arr มีต้นแบบฟังก์ชันดังนี้

```
void print_arr(int a[10]);
```

- การส่งอาร์เรย์ num ทุกอีลีเมนต์ไปให้ฟังก์ชัน print_arr สามารถเขียนได้ดังนี้

```
print_arr(num);
```

ตัวอย่างที่ 8 : การส่งอาร์เรย์ทุกอีลีเมนต์ให้กับฟังก์ชัน

- ไฟล์ arrayex8.c

```
#include <iostream>
void print_arr(int a[4]);
int main() {
    int num[4] = {5, 2, -1, 8};
    print_arr(num);
    return 0;
}

void print_arr(int a[4])
{
    for(int i = 0; i < 4; i++)
        cout << a[i];
}
```

5 2 -1 8

ตัวอย่างที่ 9 : การส่งอาร์เรย์ทุกอีลีเมนต์ให้กับฟังก์ชัน

The maximum value is 27

- ไฟล์ arrayex9.c

```
#include <iostream>
void find_max(int vals[5]); /* function prototype */
void main()
{
    int nums[5]={2, 18, 1, 27, 16};
    find_max(nums);
}

void find_max(int vals[5])    /* find the maximum value */
{
    max =vals[0];
    for (int i =1; i < 5; ++i)
        if (max < vals[i])
            max =vals[i];
    cout<<"The maximum value is" << max<<endl;
}
```

โจทย์ฝึกสมองที่ 1 : ไฟล์ arrayprac1.c

- จงเขียนโปรแกรมสำหรับรับค่าคะแนนของนักศึกษาจำนวน 10 คน และให้พิมพ์ค่าเฉลี่ยของคะแนนทั้งหมด โดยกำหนดให้ส่วนรับคะแนนจากผู้ใช้และส่วนที่แสดงค่าเฉลี่ยอยู่ใน `main` สำหรับส่วนที่คำนวณค่าเฉลี่ยให้อยู่ฟังก์ชันชื่อ `average` โดยให้ส่งอาร์เรย์ที่เก็บคะแนนทั้งของ 10 คนจาก `main` มาให้ฟังก์ชัน `average` (คะแนนสามารถเป็นทศนิยมได้)



```

#include <iostream>
float average(float num[10]);
int main()
{ int i;  float score[10],avg_score;
  for(i=0;i<10;i++)
  { Enter score[i].....
  }
  avg_score =average(.....);
  cout<< Average score is " <<.....<<endl;
  return 0;
}
float average(float num[10])
{
  .....
  return avg;
}

```

```

Enter score 1: 10
Enter score 2: 20
Enter score 3: 30
Enter score 4: 40
Enter score 5: 50
Enter score 6: 60
Enter score 7: 70
Enter score 8: 80
Enter score 9: 90
Enter score 10: 100
Average score is : 55.00

```



การแก้ไขค่าของอาร์เรย์ภายในฟังก์ชัน

ฟังก์ชันที่มีการรับพารามิเตอร์เป็นอาร์เรย์ หากมีการแก้ไขค่าภายในอาร์เรย์ดังกล่าว จะส่งทำให้อาร์เรย์ที่ถูกส่งมาเป็น อาร์กิวเมนต์ของฟังก์ชันมีการเปลี่ยนแปลงด้วย

ตัวอย่างที่ 10 : การรับพารามิเตอร์เป็นอาร์เรย์ในฟังก์ชัน

- ไฟล์ arrayex10.c

```
#include <iostream>
void edit_arr(int a[4]);
int main() {
    int num[4] = {2, 5};
    edit_arr(num);
    for(int i=0; i<4; i++)
        cout<<num[i]<<endl;
    return 0;
}
void edit_arr(int a[4]) {
    for(int i=0; i<4; i++)
        a[i] = a[i]*a[i];
}
```

4 25 0 0

```
#include <iostream>

using namespace std;

// ฟังก์ชันที่ใช้สำหรับการเรียงลำดับด้วยวิธีการ selection sort บน
// อาร์เรย์

void selectionSort(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        // ค้นหาค่าน้อยสุดในส่วนของอาร์เรย์ที่ยังไม่ถูกเรียง
        int minIndex = i;
        for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        // สลับค่าน้อยสุดที่พบกับองค์แรกในส่วนที่ยังไม่ถูกเรียง
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}
```

```
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int size = sizeof(arr) / sizeof(arr[0]);
    cout << "Array before sorting : ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }

    cout << "\nSize of array : " << size << endl;
    // เรียกใช้ฟังก์ชัน selectionSort เพื่อเรียงลำดับอาร์เรย์
    selectionSort(arr, size);

    cout << "Array after sorting: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

```
// temp / int k = arr[i] * 10;
Array before sorting : 64 25 12 22 11
Size of array : 5
Array after sorting: 11 12 22 25 64
```



Exercise:

- ทดลองแก้ตัวอย่างการเรียงข้อมูล โดยให้สามารถแสดงผลการเรียงในแต่ละรอบได้ดังนี้

Array before sorting : 64 25 12 22 11

Size of array : 5

round 1 = 11 25 12 22 64

round 2 = 11 12 25 22 64

round 3 = 11 12 22 25 64

round 4 = 11 12 22 25 64

Array after sorting: 11 12 22 25 64

- หลังจากนั้นทดลองปรับการเรียงข้อมูลจากมากไปน้อย

```
Array before sorting : 64 25 12 22 11
```

```
Size of array : 5
```

```
round 1 = 64 25 12 22 11
```

```
round 2 = 64 25 12 22 11
```

```
round 3 = 64 25 22 12 11
```

```
round 4 = 64 25 22 12 11
```

```
Array after sorting: 64 25 22 12 11 |
```

แบบฝึกหัด

- ให้นักศึกษาเขียนโปรแกรมรับค่าจำนวนนักเรียนและนำมาสร้างข้อมูล array โดยมีข้อกำหนดดังนี้
- main() รับค่านักเรียน, รับค่าชื่อและคะแนนตามจำนวนนักเรียน

เรียกใช้ฟังก์ชัน ListStudent(array)

- void ListStudent(array) แสดงผลชื่อนักเรียนและคะแนน

เรียกใช้ฟังก์ชัน Calgrade(score)

- char Calgrade(score) คำนวณเกรดตามเงื่อนไข มี 4 เกรด A B C D

คืนค่ากลับเป็นเกรด

Input Number of Student :2

Input Name :tom

Input score 1 : 50

Input Name :jerry

Input score 2 : 65

Student Name : tom Score 1 = 50 Your grade is D

Student Name : jerry Score 2 = 65 Your grade is C

Homework

ให้นักศึกษาเขียนโปรแกรมเพื่อคำนวณโบนัสของพนักงานโดยมีเงื่อนไขดังนี้

- **function main()** รับค่าจำนวนพนักงาน และรับค่าเงินเดือนของพนักงานตามจำนวน และเรียกใช้ฟังก์ชัน display เพื่อส่งค่าเงินเดือนและจำนวนพนักงานไปแสดงผล
- **function void display(int salary[],int num)** แสดงผลเงินเดือน และโบนัสของพนักงานทั้งหมด โดยการแสดงผลโบนัสให้เรียกใช้ฟังก์ชัน cal_bonus เพื่อให้คำนวณเงินโบนัสของพนักงานแต่ละคน
- **function int cal_bonus(int salary)** รับค่าเงินเดือนแต่ละคน (ไม่ใช่ array เพื่อคำนวณโบนัสโดยมีเงื่อนไข คือ โบนัส = เงินเดือน * 2 และส่งค่า bonus กลับไปยัง function display

```

Enter Number of person : 2
Enter the salary 1 : 50000
Enter the salary 2 : 30000

-----
There are 2 persons.
The Salary for person 1 = 50000 and Bonus = 100000
The Salary for person 2 = 30000 and Bonus = 60000
-----
Press any key to continue . . .
  
```